Recitation 4

Recitation Overview

- Bash – in depth review of a sample script in /share/recitations/recitation04/count-files.sh – copy the content of that folder into your homedirs for editing

```
mkdir recitation04
cd recitation04
cp -r /share/recitations/recitation04/* .
```

Counting files recursively

- Task: count the number of readable files that are located in a given directory and all its subdirectories (recursively).

Counting files recursively - strategy

- Validate the input
- Count the readable files in the current directory
- Get all subdirectories in the current directory
- Run the script on all the subdirectories
- Breaking condition:
  - No subdirectories (result = readable number of files in the parent directories)

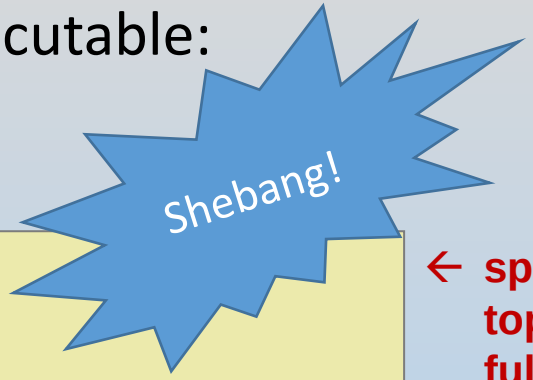
Shell scripts as self-contained programs

If you write a script to be executed in its own shell, then you can specify which scripting program to run it on, and then invoke the script as an executable:

`script_file:`

```
#!/bin/bash

…
```



← **special comment line at top of file specifying the full path of shell program**

```
==> chmod +x sample_script
==> ./script_file
```

← **ensure executable permissions**

← **run script**

Input arguments

- Arguments may be passed onto a script as a list of space separated strings

```
==> script_path arg1 arg2 …
```

- The i^{th} argument can be accessed by `$i` (e.g., `$1`, `$2`,…)
- Number of arguments is kept in `$#`
- Entire list of arguments is kept in `$*`
- The name of the script itself is held by `$0`

Bash if statements

Syntax:

mandatory spaces here

```
if [ conditional ]; then
   ...
elif [ conditional ]; then
   ...
else
   ...
fi
```

← 'then' in separate line (or after ';')

optional branching conditionals

← closes if statement

Examples of conditional statements:

```
# string comparisons
if [ "$name" != "Hillary Clinton" ]  . . .

# number comparisons (+ composite conditions)
if [ "$num" -lt 100 ] && [ "$num" -gt 54 ]  . . .

# file system queries
if [ ! -d "$dir" ]  . . .
```

➔ More details in the bash control flow guide on Piazza.

Bash loops

For loop syntax:

```
for <iterator> in <listVals>
do
    ...
done
```

iterator variable

space separated list

← 'do' in separate line (or after ';')

← closes the for loop

While loop syntax:

```
while [ conditional ]
do
    ...
done
```

mandatory spaces here

← 'do' in separate line (or after ';')

← closes the for loop

- The **`while`** conditional is formatted the same way as the **`if`** conditional.

➔ More details in the bash control flow guide on Piazza.

Script output

- Main program output is simply printed to **`stdout`**
- The output is captured by an executing script by using backquotes (` `)
- Example (in **`compact-files-prog.sh`**):

```
...
my_files=`ls -l $dir | . . . `
...
```

Alternatively, if the piped line is implemented in a separate (executable) script **`list-files.sh`** (as suggested in slide #10):

```
...
files=`./list-files.sh`
...
```

Exit status

- Main program output is simply printed to `stdout`
- Programs typically return some value to the system (e.g., final return from main() in C)
- Scripts return a value through the '`exit`' command. The value of the previous command's exit status is held by variable `$?`.